

A Storage-Centric Analysis of MapReduce Workloads: File Popularity, Temporal Locality and Arrival Patterns

Cristina L. Abad^{*†‡}, Nathan Roberts[†], Yi Lu^{*} and Roy H. Campbell^{*}

^{*}University of Illinois at Urbana-Champaign; [†]Yahoo! Inc.

^{*}{cabad,yilu4,rhc}@illinois.edu; [†]{nroberts}@yahoo-inc.com

Abstract—A huge increase in data storage and processing requirements has lead to *Big Data*, for which next generation storage systems are being designed and implemented. However, we have a limited understanding of the workloads of Big Data storage systems. We consider the case of one common type of Big Data storage cluster: a cluster dedicated to supporting a mix of MapReduce jobs. We analyze 6-month traces from two large Hadoop clusters at Yahoo! and characterize the file popularity, temporal locality, and arrival patterns of the workloads. We identify several interesting properties and compare them with previous observations from web and media server workloads. To the best of our knowledge, this is the first study of how MapReduce workloads interact with the storage layer.

Index Terms—Big Data; MapReduce; HDFS; access patterns

I. INTRODUCTION

Due to an explosive growth of data in the scientific and Internet services communities and a strong desire for storing and processing the data, next generation storage systems are being designed to handle peta and exascale storage requirements. As Big Data storage systems continue to grow, a better understanding of the workloads present in these systems becomes critical for proper design and tuning.

We analyze one type of Big Data storage cluster: clusters dedicated to supporting a mix of MapReduce jobs. Specifically, we study the file access patterns of two multi-petabyte Hadoop clusters at Yahoo! across several dimensions, with a focus on popularity, temporal locality and arrival patterns. We analyze two 6-month traces, which together contain more than 940 million creates and 12 billion file open events.

We identify unique properties of the workloads and make the following key observations:

- Workloads are dominated by high file churn (high rate of creates/deletes) which leads to 80% – 90% of files being accessed at most 10 times during a 6-month period.
- There is a small percentage of highly popular files: less than 3% of the files account for 34% – 39% of the accesses (opens).
- Young files account for a high percentage of accesses, but a small percentage of bytes stored. For example, 79% – 85% of accesses target files that are most one day old, yet add up to 1.87% – 2.21% of the bytes stored.
- The observed request interarrivals (opens, creates and deletes) are bursty and exhibit self-similar behavior.
- The files are very short-lived: 90% of the file deletions target files that are 22.27 mins – 1.25 hours old.

Derived from these key observations and a knowledge of the domain and application-level workloads running on the clusters, we highlight the following insights and implications to storage system design and tuning:

- The peculiarities observed are mostly derived from short-lived files and high file churn.
- File churn is a result of typical MapReduce workflows: a high-level job is decomposed into multiple MapReduce jobs, arranged in a directed acyclic graph (DAG). Each of these (sub-)jobs writes its final output the storage system, but the output that interests the user is the output of the last job in the graph. The output of the (sub-)jobs is deleted soon after it is consumed.
- High rate of change in file population prompts research on appropriate storage media and tiered storage approaches.
- Caching young files or placing them on a fast storage tier could lead to performance improvement at a low cost.
- “Inactive storage” (due to data retention policies and dead projects) constitutes a significant percentage of stored bytes and files; timely recovery of files and appropriate choice of replication mechanisms and media for passive data can lead to improved storage utilization.
- Our findings call for a model of file popularity that accounts for a very dynamic population.

To the best of our knowledge, this is the first study of how MapReduce workloads interact with the storage layer.

We provide some background on MapReduce clusters in § II, and a description of our datasets in § III. In § IV we provide a characterization of the storage workloads of two MapReduce clusters. The related work is discussed in § V. Finally, in § VII we conclude.

II. BACKGROUND: MAPREDUCE CLUSTERS

MapReduce clusters [1, 2] offer a distributed computing platform suitable for data-intensive applications. MapReduce was originally proposed by Google and its most widely deployed implementation, Apache Hadoop, is used by many companies including Yahoo!, Facebook and Twitter.

MapReduce uses a divide-and-conquer approach in which input data are divided into fixed size units processed independently and in parallel by *map* tasks, which are executed distributedly across the nodes in the cluster. After the *map* tasks are executed, their output is shuffled, sorted and then processed in parallel by one or more *reduce* tasks.

To avoid network bottlenecks due to moving data in/out of the compute nodes, a distributed file system typically co-exists with the compute nodes (e.g., HDFS [3] for Hadoop).

[†] Also affiliated with Facultad de Ingeniería en Electricidad y Computación (FIEC), Escuela Superior Politécnica del Litoral (ESPOL), Campus Gustavo Galindo, Km 30.5 Vía Perimetral, Guayaquil-Ecuador.

TABLE I

SOME RELEVANT STATISTICS OF THE DATASETS USED IN THIS PAPER; $t_0 = 2011/06/08$ AND $t_1 = 2011/12/9$; 1 M: 1 MILLION, 1 B: 1000 M.

	Cluster size	Used storage at:		Files in namespace at:		Creates during	Opens during
		t_0	t_1	t_0	t_1	$(t_0 - t_1)$	$(t_0 - t_1)$
PROD	4146 nodes	3.83 PB	3.93 PB	51.39 M	54.22 M	721.66 M	9.71 B
R&D	1958 nodes	2.95 PB	3.63 PB	38.26 M	51.37 M	227.05 M	2.93 B

MapReduce clusters have a master-slave design for the compute and storage systems. The master file system node handles the metadata operations, while the slaves handle the read/writes initiated by clients. Files are divided into fixed-sized blocks, each stored at a different data node. Files are typically read-only, but appends may be performed in some implementations. The storage component of these clusters uses a configurable number of replicas per file (three by default). This mechanism is meant to be used for improved availability, fault-tolerance and load balancing.

In this paper, we refer to the components of the distributed file system using the HDFS terminology, where *name node* refers to the master node and *data node* refers to the slave.

III. DATASET DESCRIPTION

We analyzed 6-month *namespace metadata traces*¹ from two Hadoop clusters at Yahoo!:

- *PROD*: 4100+ nodes, using the Hadoop Distributed File System (HDFS). Production cluster running pipelines of data-intensive MapReduce jobs like processing advertisement targeting information.
- *R&D*: 1900+ HDFS nodes. Research and development cluster with a superset of search, advertising and other data-intensive pipelines.

The jobs in PROD are typically batch jobs that need to run on a regular basis (e.g., hourly jobs, daily jobs, weekly jobs). R&D is used to test some of the jobs running in PROD and jobs to be moved to PROD in the future; in R&D there is less emphasis in timeliness. Additionally, R&D is also used to run somewhat interactive, data-analytics/Business Intelligence queries. Both clusters run plain MapReduce jobs, as well as MapReduce workflows generated by Apache Pig (a dataflow language that is compiled into MapReduce) and by Apache Oozie (a workflow scheduler for Hadoop).

The *namespace metadata traces* analyzed consist of a snapshot of the namespace on June 8th, 2011 (t_0), obtained with Hadoop's Offline Image Viewer tool, and a 6-month access log trace (Jun. 9th, 2011 through Dec. 8, 2011), obtained by parsing the name node audit logs. For some of our analysis, we also processed a snapshot of the namespace taken on Dec. 9, 2011 (t_1). For simplicity, we refer to the log with the set of events (open, create, etc.) as a **trace**. Figure 1 shows an example of a typical entry in the HDFS audit logs (trace). Table I provides a summary of these traces.

Limitations of the traces: (i) millisecond granularity (a higher granularity would be desirable), and (ii) do not include

¹We define a **namespace metadata trace** as a storage system trace that contains a snapshot of the namespace (file and directory hierarchy) and a set of events that operate atop that namespace (e.g., open a file, list directory contents) [4]. These traces can be used to evaluate namespace management systems, including their load balancing, partitioning, and caching components.

```
2012-5-18 00:00:00,134 INFO FSNamesystem.audit:
ugi=USERID ip=<IP-ADDRESS> cmd=open
src=/path/to/file dst=null perm=null
```

Fig. 1. HDFS name node log record format example.

I/O information. The latter precludes us from knowing the size of a file once its created. While we can obtain the size of the files in a snapshot (say, at time t_0), Yahoo! only keeps record of daily snapshots making it impossible to know the size of files created and deleted in between snapshots. Issue (ii) also precludes us from knowing how many bytes are read upon an open event. While MapReduce jobs typically will read a file all at once, we cannot do any analysis that requires certainty in the knowledge of the number of bytes read.

IV. ANALYSIS OF TWO MAPREDUCE WORKLOADS

We present an analysis of the data (file) access patterns present in the traces described in § III and discuss the implications to storage design. Other characteristics of the workloads, not directly related to the access patterns, are also presented to help provide a broader characterization of the workloads and which may be of interest to other researchers.

We highlight some of the most important insights using italics; for example, *IO: Insight about workload*.

A. File popularity

Figure 2 shows the Complementary Cumulative Distribution Function (CCDF) of the file accesses (opens), for both clusters, for different periods of time: first day of the trace, first month of the trace and full six-month trace. The CCDF shows $P(X \geq x)$, or the cumulative proportion of files accessed x or more times. The dashed line shows the best Power Law fit for the tail of the distribution. Files not accessed during the trace were ignored for these plots; a brief discussion on “inactive” storage is presented later in this section.

TABLE II

BEST FIT OF FILE ACCESS FREQUENCY (FIGURE 2) TO A POWER LAW. α : SCALING PARAMETER, x_{min} : LOWER BOUND OF POWER-LAW BEHAVIOR.

	α	x_{min}
PROD, 1-day trace	2.22	464
PROD, 1-month trace	2.47	770
PROD, 6-month trace	2.99	937
R&D, 1-day trace	2.22	1
R&D, 1-month trace	2.11	189
R&D, 6-month trace	2.36	325

Since file access patterns in other workloads exhibit Power Law behavior (or Zipf Law if ranked data is analyzed), we provide the results of the best fit of the tail of the distribution to a Power Law. To find the best fit, we apply the methodology (and toolset) described by Clauset et al. [5]. Results are shown

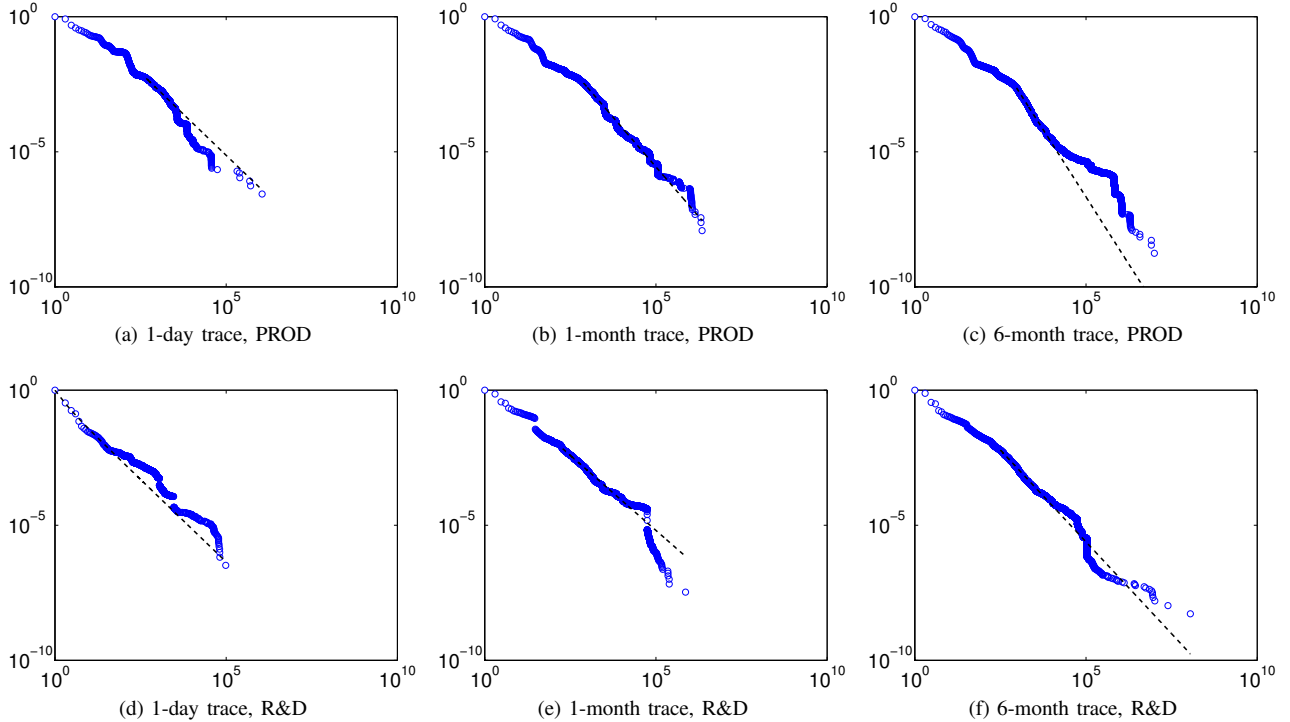


Fig. 2. Complementary Cumulative Distribution Function (CCDF) of the frequency of file accesses (opens), for increasingly larger traces. The CCDF shows $P(X \geq x)$, or the cumulative proportion of files accessed x or more times in the trace. The dashed line shows the best Power Law fit for the tail.

in Figure 2 and Table II. The latter shows the Power Law scaling parameter (α) and x_{min} , the value where the fitted tail begins. x_{min} is chosen so that the Kolmogorov-Smirnov goodness-of-fit test statistic (D)—which is the maximum difference between the two CDF curves—is minimized.

How popular are the most popular files? With Power Law tail behavior, a small percentage of the files typically account for a large percentage of the accesses. For example, for enterprise media server workloads the authors in [6] found that between 14% – 30% of the files account for 90% of the media sessions. In our analysis, we found the workloads to be less skewed towards popular files (see Figure 2). Specifically, for the case of the 6-month traces, 22% – 29% (R&D and PROD, respectively) of the distinct files accessed in the 6-month period account for 90% of the accesses. If we instead calculate these values as a percentage of the total number of files stored at time t_1 (see Table III), the percentages increase to 88.89% – 304.61% (R&D and PROD, respectively). The percentage above 100% is an artifact of many files in that 90% that were deleted during the 6-month period. None of these two ways of calculating the percentage provide an accurate view of the popularity of the files. The second approach (dividing by the number of files stored at t_1) is obviously wrong, as it may lead to percentages above 100%. It may be less obvious, however, why dividing by the total number of distinct files in the trace is problematic too.

The problem with dividing by the number of distinct files that were accessed at least once during the traces is that this number of files never exists in the system at one time. From Table I, we have that the number of creates during $t_0 - t_1$ is one order of magnitude larger than the number of files at t_1 ;

most of the files are deleted soon after they are created.

TABLE III
MOST POPULAR FILES STATISTICS (6-MONTH TRACES). REFER TO THE TEXT FOR AN EXPLANATION OF WHY SOME VALUES ARE ABOVE 100%.

	Files accounting for up to 80% / 90% of the accesses
<i>As % of distinct files in trace</i>	
PROD	12.88% / 29%
R&D	5.39% / 22%
<i>As % of files in namespace (t_1)</i>	
PROD	135.35% / 304.61%
R&D	19.88% / 88.89%

TABLE IV
INFREQUENTLY ACCESSED FILES STATISTICS, AS A PERCENTAGE OF THE NUMBER OF DISTINCT FILES IN THE 6-MONTH TRACES.

Trace	Files accessed		
	1 / up to 5	up to 10 times	
PROD	15.03%	68.40%	80.98%
R&D	23.66%	84.25%	90.08%

To understand why these short-lived files constitute a problem when quantifying the popularity of files, we flip the question and analyze the *unpopular* files: At the other end of the distribution, we find a high percentage of unpopular (or rarely accessed) files. For example 80 – 90% of the files are accessed no more than 10 times during the full 6-month period (Table IV). An important contributor to the long tail of unpopular files is the high percentage of short lived files (details in § IV-B and IV-D).

Another study from a different Big Data workload (6-month 2009 trace from Bing’s Dryad cluster [7]) shows similar findings for unpopular files: 26% of the files were accessed at

most once, $\approx 92\%$ at most 5, and $\approx 97\%$ at most 10 times. On the other hand, a study on media server workloads [6] found that 47 – 59% of the files were requested up to 10 times.

Table IV does not include information about files never accessed in the 6-month period. The files that exist at t_1 and were not accessed (open) during the 6-month period constitute *inactive storage* and account for a high percentage of the files (51% – 52%) and bytes stored (42% – 46%) at t_1 ². Of those files, 33% (R&D) – 65% (PROD) (15% – 26% of bytes) existed at t_0 ; the rest were created between t_0 and t_1 . There are two main reasons for having inactive storage: (i) data retention policies (i.e., minimum period of time during which a copy of the data needs to be stored), and (ii) dead projects, whose space is manually recovered (freed) from time to time (e.g., when the free space on the cluster falls below some limit).

Automatic dead project space recovery, intelligent replication/encoding [8] and tiered-storage mechanisms can reduce the impact of inactive storage. For example, using erasure encoding instead of replication and a slower (and thus, cheaper) storage tier for passive data can lead to a significant cost reduction while still fulfilling data retention policies.

I1: Inactive storage constitutes a significant percentage of stored bytes and files; timely recovery of files and appropriate choice of replication mechanisms and media for passive data can lead to improved storage utilization and reduced costs.

We use the information gathered from our analysis on the unpopular files to go back to the question of *how popular are the popular files*. Recall that, using the approach of dividing the number of accesses (frequency) by the number of distinct files that were accessed at least one during the trace, we found that 29% of the files in PROD and 22% of the files in R&D account for 90% of the accesses. However, we also know from Table IV, that 81% (PROD) – 90% (R&D) of the files are accessed no more than ten times in the full 6-month period. It should now be more clear that these metrics are misleading and can lead to confusion. To be specific, consider the case of R&D: $22\% + 90\% > 100\%$; this means that these two groups are not mutually exclusive, and some files are counted in both groups. In other words, some files in the “very popular” group have been accessed at most 10 times during the 6-month period! While the term “very popular” is subjective, we believe it is unreasonable to apply the tag to a file that has been accessed, on average, less than twice a month.

I2: A model of file popularity with a static number of files is inadequate for workloads with high file churn.

Are some files extremely popular? The answer depends on how we define *extremely*. Since the issues in the percentages discussed before come from the difficulty in deciding what 100% means in a period during which the population has changed significantly, we now use frequency counts instead of percentages. We did an analysis on the files that were accessed at least 10^2 times and those accessed at least 10^3 times (during the 6-month period). For PROD, 117505 files were accessed at least 10^3 times, and 8239081 files were accessed at least 10^2 times, constituting 2.17% and 15.2% of the files in the

namespace at t_1 . For R&D, 243316 files were accessed at least 10^3 times, and 3199583 files were accessed at least 10^2 times, constituting 0.47% and 6.23% of the files in the namespace at t_1 . Finally, if we sum all the accesses to all the files that are in the “ 10^3 or more accesses” group, we have that 34% of the open events in PROD and 39% of the open events in R&D targeted the top 2.17% and top 0.47% files respectively.

I3: Workloads are dominated by high file churn (high rate of creates/deletes) which leads to 80% – 90% of files being accessed at most 10 times during a 6-month period; however, there is a small percentage of highly popular files: less than 3% of the files account for 34% – 39% of accesses.

B. Temporal locality

Prior studies have noted that MapReduce workloads tend to concentrate most of the accesses to a file in a short period after the file is created [8, 9]. This temporal locality can be captured with the distribution of the *age of a file at the time of access* (AOA). Basically, for each access to a file, we calculate how old the file is at that moment. To do this, we need to know *when* each file was created. We obtain this information from: (a) the namespace snapshot, for those files that were created before the trace was captured, and (b) from the create events present in the trace. Since the HDFS audit logs contain the full path + name of each file instead of a unique identifier for the file, we also kept track of file renames to have an accurate record of the creation stamps.

Figure 3 shows the AOA for traces of varying length. We observe some changes in the distribution, due to the non-stationary nature of the workload (monthly changes in Fig. 4).

Let’s consider the AOA distribution during the 6-month period (see Figures 3 and 4 and Table V). In PROD, most accesses target very young files: 50% of the accesses (open events) target files that are at most 407 seconds old. In R&D, files remain popular for a longer (but still short) period: 50% of the accesses target files that are at most 33 minutes old. The difference can be explained by understanding the characteristics of the workloads of these two clusters: the jobs in PROD process data recently recorded or generated (for example, the day’s log of user clicks), and they are *not* interactive; on the other hand, some of the jobs on R&D tend to be of the interactive, data-analytics/Business Intelligence type. The latter type of jobs are more user-driven, which accounts for the higher latency between accesses (as opposed to the highly automated batch jobs in PROD).

We notice the closeness of the 90% percentile in Table V (PROD) to the 1-week mark and ask this question: *What percentage of accesses target files that are at most one week old?* The answer, is surprisingly close for both clusters: 90.31% (PROD) and 86.87% (R&D). To provide some perspective, a media server study [6] found that the first five weeks of a file’s existence account for 70 – 80% of their accesses.

Regarding accesses to very young files, 29% – 30% of accesses target files that are at most 2 minutes old. We believe this is an effect of the typically short duration of the MapReduce jobs on these clusters. For example, during the same 6-month period, 34.75% – 57.46% (PROD and R&D)

²To avoid a distortion in the results due to transient files at t_1 , we ignored those files that do not exist at $t_1 + 1$ day. Otherwise, the inactive storage percentage would go up to 57% – 65% of the files at t_1 .

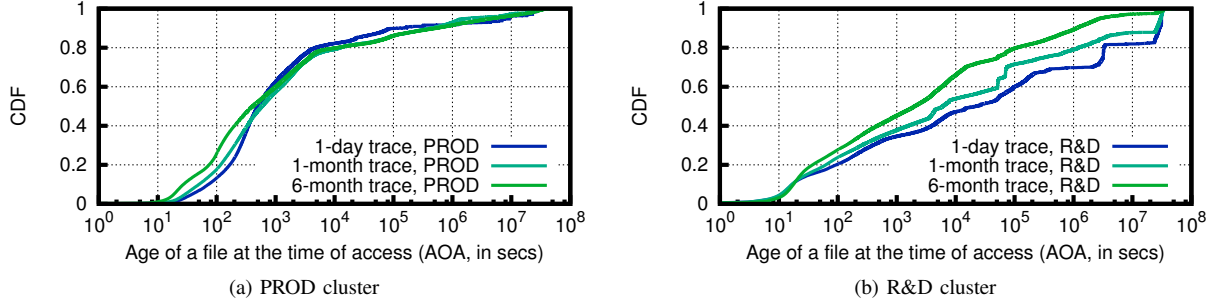


Fig. 3. Cumulative distribution function (CDF) of the age of a file at each time of access (access = open), for increasingly larger traces.

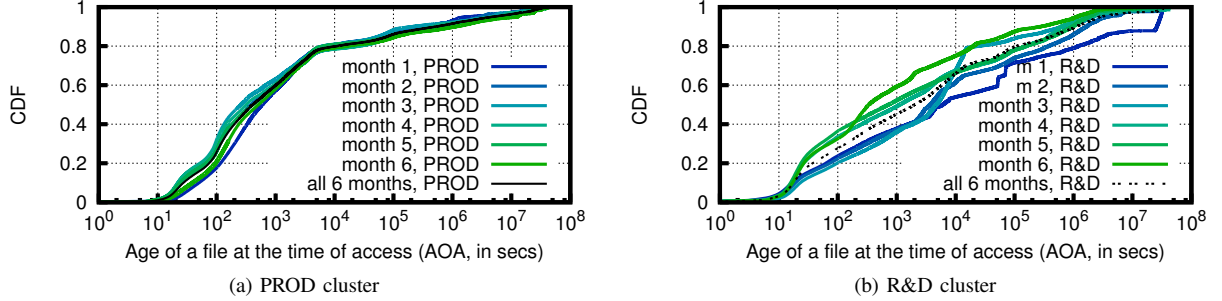


Fig. 4. Cumulative distribution function (CDF) of the age of a file at each time of access (access = open), for each of the months analyzed in this paper.

of the successful jobs had a total running time of 1 minute or less (including the time waiting on the scheduler queue)³.

The access skew towards young files can be exploited by caching strategies and tiered-storage designs. For example, 78.91% (R&D) to 85.41% (PROD) of the accesses target files that are most 1 day old. However, at one particular point in time (say, at t_1), the files these files constitute 1.01% (R&D) to 3.67% (PROD) of the files and 1.87% (R&D) to 2.21% (PROD) of the bytes stored. Caching these files or storing them on a tier faster than disk would improve performance.

14: Young files account for a high percentage of accesses but a small percentage of bytes stored. Caching or placing these files on a fast storage tier could improve performance at a low cost.

TABLE V
AGE OF A FILE AT A TIME OF ACCESS (AOA) STATISTICS (6-MONTH TRACE). FULL DISTRIBUTION OF ACCESSES SHOWN IN FIGURE 3.

Cluster	50%	80%	90%
PROD	407.80 secs	3.06 hours	6.11 days
R&D	33.53 mins	1.25 days	13.06 days

For a particular cluster, there could be consistent changes in the AOA distribution as time progresses. For example, the curve could slowly start moving to the left or to the right every month. This behavior can be seen in R&D but not in PROD (Figure 4). We believe the difference can be explained by the nature of the workloads. Jobs in PROD are repetitive across days, weeks and months. On the other hand, jobs in R&D are more dynamic and user-driven, with changes influenced by short-term user needs.

³We obtained these percentages by analyzing the job tracker's (Hadoop's central scheduler) logs.

C. Request arrivals

We analyze the arrivals of the operation requests at the namespace server. Figure 5 shows the cumulative distribution function (CDF) of the interarrivals of the different operations (open, create, delete) in the 6-month traces⁴. As expected, the open operations are more frequent than the creates and deletes, but it is interesting to observe the high rate at which files are created (and deleted). For example, in PROD 36.5% of the create operations have an interarrival of 1 millisecond or less.

To model the interarrivals, one can fit the observed interarrivals to a known distribution and use this distribution to sample the interarrivals, or use the empirical distribution described by the CDF if no good fit is found. However, defining the interarrivals by using a CDF (empirical or fitted) implicitly assumes independence of the random process.

Interarrival times may present autocorrelations; for example, previous studies on Ethernet and Web traffic have shown that they are often bursty and even self-similar⁵ [10, 11]. Self-similar arrivals have implications to the performance of the server queues (in this case, the namespace metadata server): unlike Poisson arrivals, self-similar arrivals exhibit burstiness that may itself be bursty and requests may backlog in the queue of the server if it is not able to serve them fast enough. In other words, the queue length distribution of a self-similar process decays more slowly than that of a Poisson process [12].

Figure 6 shows the burstiness of the arrivals of the open events, at different time scales; the arrivals exhibit burstiness, even at increasingly larger aggregation periods, a sign of self-

⁴We did not analyze the arrivals of the other types of operations (e.g., listStatus, mkdir, etc.) because those operations are related to the namespace (and not the data), and are thus out of the scope of this paper.

⁵A *self-similar* process behaves the same when viewed at different scales. In this context, the request arrivals are bursty at different time scales.

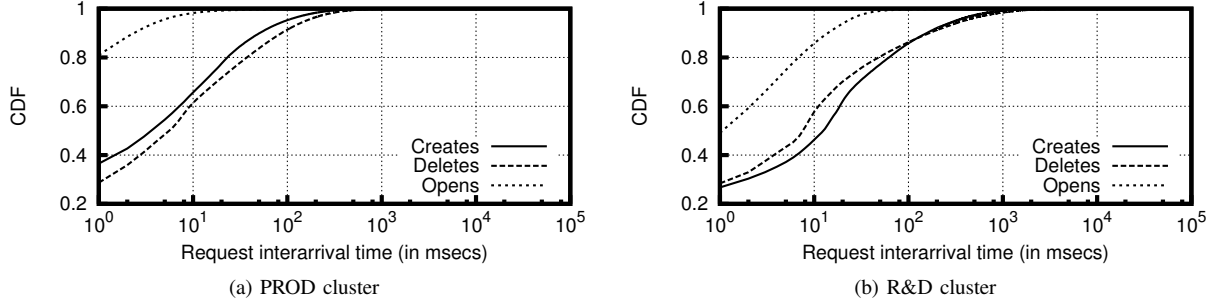


Fig. 5. Cumulative distribution function (CDF) of the interarrival times of the open, create and delete events, during the 6-month period.

similar behavior. Create and delete arrivals are also bursty; we do not include those graphs due to space constraints.

To test for self-similarity, we use two methods used by previous literature [11]: the *variance-time plot* and the *R/S plot*, and estimate the *Hurst parameter* (H). The *Hurst parameter* provides a measure of the burstiness of a time series (in this case, the counting process of the arrivals); more formally, it expresses the speed of decay of the series’ autocorrelation function [11]. The results are shown in Figure 7. Using the *variance-time plot* method we estimate $H = 0.937$ (PROD) and $H = 0.902$ (R&D); using the *R/S plot* method we estimate $H = 0.8136$ (PROD) and $H = 0.9355$ (R&D). These results correspond to the first hour of the 6-month trace, with an aggregation period m of 60 msecs (e.g., we counted the arrivals for each of the 60,000 non-overlapping time slots of 60 milliseconds each during that hour). In all cases, $1/2 < H < 1$, which implies that the time series is self-similar with long-range dependence (i.e., the autocorrelation function decays slower than exponentially).

Figure 8 shows the estimation of H during the same hour, for varying aggregation periods (m). Figure 9 shows the changes in H for every hour of a 24-hour period. For all these cases, the estimation of H is consistently in the range $1/2 < H < 1$. The variability in the value of H in Figure 9 is due to load changes during different hours of the day, with a smaller H during less busy hours [11]. The value of H in PROD is much more stable because this cluster is not affected by user working hours. The load of R&D is user-driven and, thus, more variable; the spike at the end of the day results from jobs scheduled during low usage hours.

To accurately model the arrivals of the requests, we should preserve the interarrivals *and* the burstiness; for example, by using Markovian Arrival Processes [13] which can capture the autocorrelations present in the trace (ordering) with a minor loss in accuracy in the distribution fitting.

15: The request interarrivals are bursty and exhibit self-similar behavior.

The arrivals of create and delete operations are also bursty and self-similar. Table VI shows the estimate of H for the arrival processes of creates and deletes ($m = 60$ msecs).

D. Life span of files

Figure 10 shows the distribution of the age of a file at the time of deletion (how old are files when deleted). The files are

TABLE VI
ESTIMATION OF H FOR CREATES AND DELETES, DURING THE FIRST HOUR OF THE 6-MONTH TRACES AND $m = 60$ msecs.

	PROD	R&D
<i>Creates</i>		
Variance-time plot method	0.884	0.928
R/S plot method	0.9919	0.9696
<i>Deletes</i>		
Variance-time plot method	0.867	0.931
R/S plot method	0.8832	0.8716

short-lived: 90% of deletions target files that are 22.27 mins (PROD) to 1.25 hours (R&D) old (see Table VII). In more traditional workloads like that of media servers, files have a longer life span: a study [6] found that 37% – 50% of media files “live” (calculated as time between the first and last access) less than a month, a lower bound on the real life span.

TABLE VII
AGE OF A FILE AT THE TIME OF DELETION (AOD) STATISTICS (6-MONTH TRACE). FULL DISTRIBUTION OF ACCESSES SHOWN IN FIGURE 10.

Cluster	50%	80%	90%
PROD	117.1 secs	453.36 secs	22.27 mins
R&D	238.51 secs	26.61 mins	1.25 hours

In MapReduce workloads, many jobs are actually composed of several smaller MapReduce jobs executed in series (or as part of a Directed Acyclic Graph (DAG) workflow); the output of one job is the input of the next one. These files are not needed after the next job finishes and are thus deleted soon after consumed, leading to short-lived files and high file churn. They should be thought of as “intermediate” data that is temporarily written into the distributed storage system⁶.

16: The files are very short-lived: 90% of deletions target files that are 22.27 mins – 1.25 hours old.

For modeling and workload generation purposes it may be useful to know the age of the files that exist in the system at t_0 [4]. Figure 11a shows this distribution. Note that in R&D—where there is less emphasis on processing “fresh” data—the stored files tend to be older: the file age median in R&D is 111.04 days vs. 60.85 days in PROD.

E. File sizes

Figure 11b shows the sizes of the files in the namespace at t_0 , using 2MB bins. This information is useful for modeling

⁶Not to be confused with the intermediate data that is generated by the map phase and consumed by the reduce phase, which is not written to HDFS.

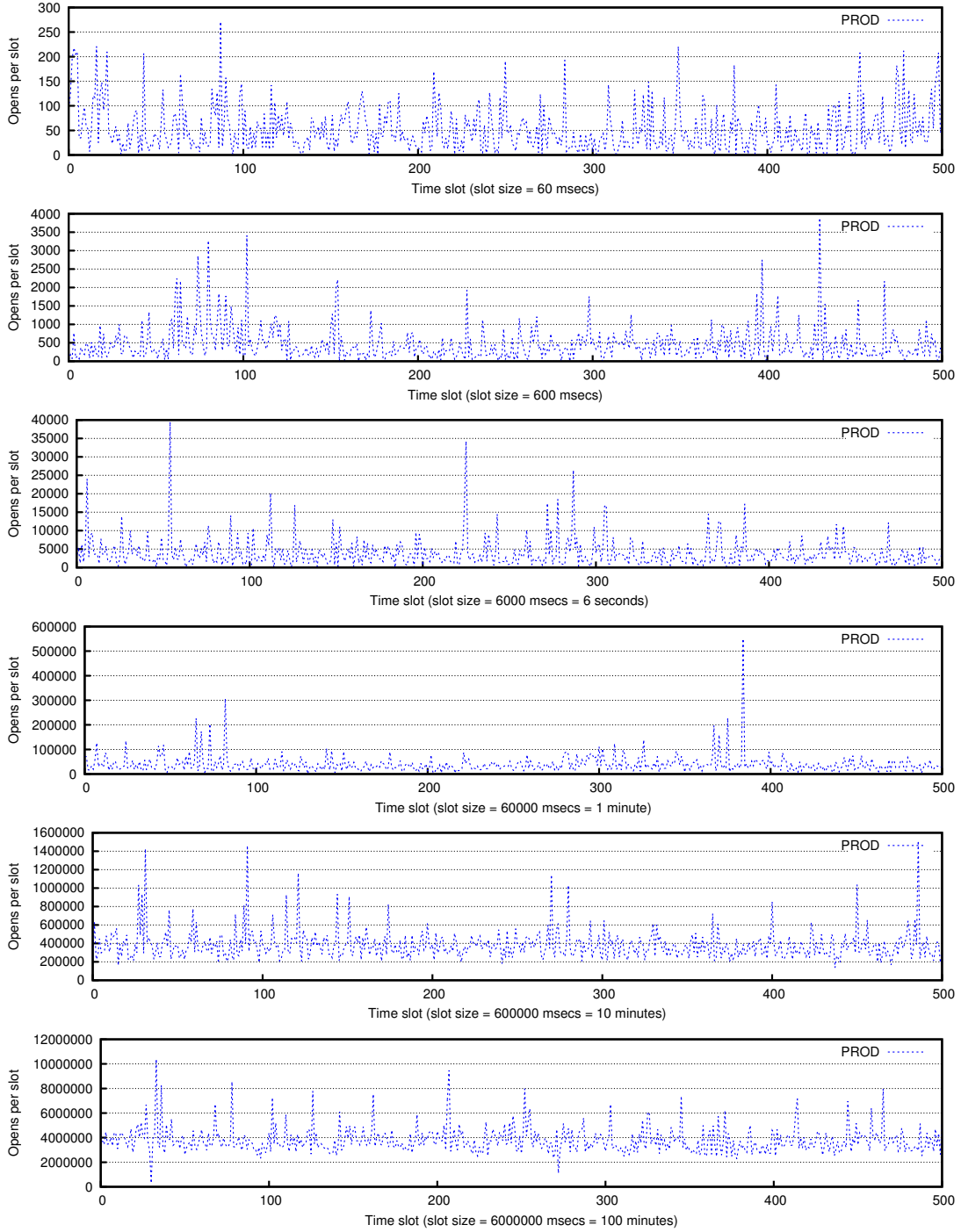


Fig. 6. Number of opens per time slot, for the PROD cluster. The arrivals are bursty at different timescales.

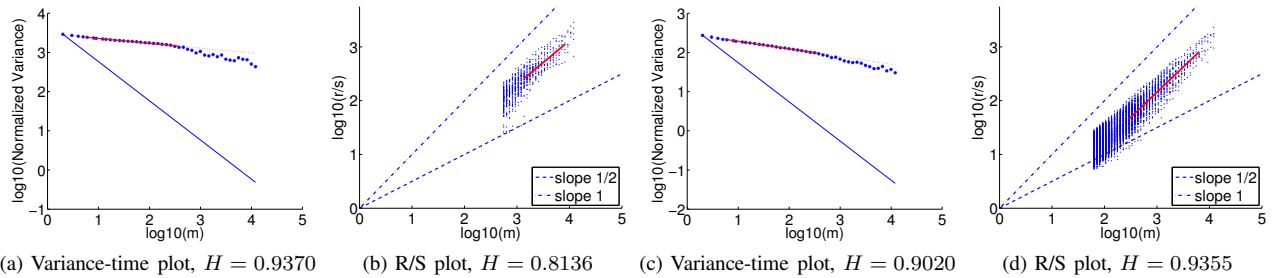


Fig. 7. Graphic analysis of the self-similarity in the arrivals of the open requests and Hurst parameter (H) estimate, for PROD (a-b) and R&D (c-d), during the first hour of the trace, calculated with an aggregation period $m = 60 \text{ msec}$.

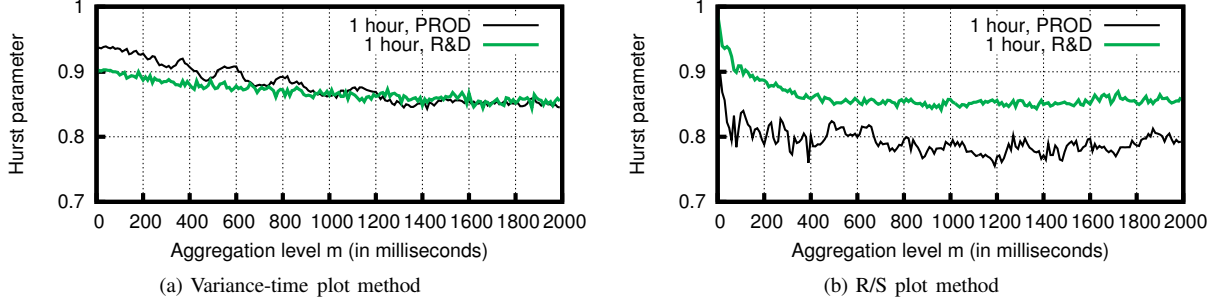


Fig. 8. Estimation of H with varying aggregation period m , for both clusters (1st hour of the 6-month period).

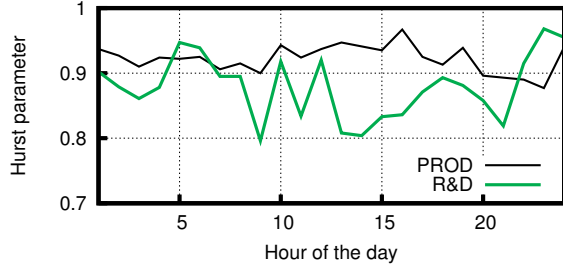


Fig. 9. Estimate of H during a consecutive 24-hour period (1st day of 6-month trace); there is some change in burstiness during the day.

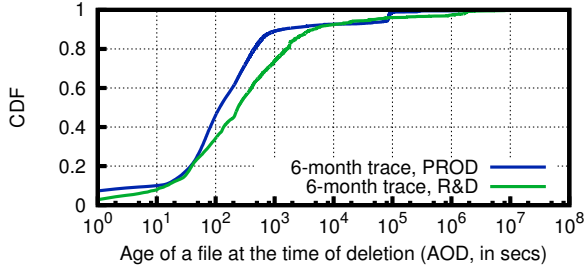


Fig. 10. Age of file at the time of deletion (AOD), for files deleted during trace. This distribution encodes information about the life span of files.

the initial status of the namespace [4]. We ignored the files with size 0, which constitute 4% (PROD) and 11% (R&D) of the files in the namespace, because they increase significantly the percentage of the first bin and would not allow us to know the exact percentage of very small files. The zero-sized files would otherwise account for 99.9% (PROD) and 99.98% (R&D) of the files in the 0-2MB bin.

Why is there such a high number of zero-sized files? Hadoop uses some zero-sized files as flags (e.g., signaling that a job has finished). Flag files constitute 35% (PROD) and 5.6% (R&D) of the zero-sized files. However, the biggest contributor to zero-sized files are empty job output files. For example, out of 100 reducers, some percentage of those may not generate any data, but will nevertheless generate an empty output file named `part-X` where X is a number that identifies the reducer (or mapper) that generated the file. These files account for 52.96% (PROD) and 93.71% (R&D) of the zero-sized files at t_0 .

Is there a correlation between the size of a file and its popularity? We calculated the *Pearson's correlation coefficient* between the size of a file and its popularity, using both the

number of accesses and the file rank as a metric of popularity, and using two types of binning approaches for the data: fixed-size bins of 2MB and bins with logarithmically ($\ln$) larger width. We found no strong correlation between the size of a file and its popularity, although a medium correlation is found in the R&D cluster, when using the file rank as the popularity metric (Table VIII). However, as indicated in § III, we cannot know the size of files created and deleted in between the daily snapshots, so those files were ignored in this analysis. Sampling prior to a correlation analysis yields accurate results if there is no bias in the sampling of the data. In this case, we know that there is a bias against short-lived files but cannot tell if there is a bias in the file sizes. It is possible that the correlation results would be different if all files had been considered, so this issue warrants further analysis in the future.

I7: There appears to be no strong correlation between the size of a file and its popularity.

TABLE VIII
PEARSON'S CORRELATION COEFFICIENT BETWEEN FILE SIZE AND POPULARITY (6-MONTH TRACES); $|1| \Rightarrow$ STRONG CORRELATION.

	PROD	R&D
Access count – 2MB bins	0.0494	0.0299
Access count – $\ln$ bins	-0.0014	0.0063
File rank – 2MB bins	0.0144	-0.3593
File rank – $\ln$ bins	-0.2297	-0.4048

Note that a correlation between size and popularity was found in a Big Data workload from Bing's Dryad [7]. On the other hand, a study on web server workloads did not find a correlation between these dimensions [14]. A lack of correlation would have negative implications to caching; however, the previous observations on file life span and churn could be used to design effective tiered storage systems.

F. File extensions

We calculated the percentage of files that have an extension; for this purpose we used the Java RegEx “`\. ([^.]*)$`” on the full path, and excluded those files for which (a) the extension had more than 5 characters, and (b) had an extension that consisted of only numbers. Using this approach, we found that 39.48% – 54.75% (R&D and PROD, respectively) of the files have an extension. Table IX shows the top eight extensions and their percentages. The three most common file extensions in these clusters are bz2, gz and xml (note that bz2 and gz are compression formats supported by Hadoop). Using

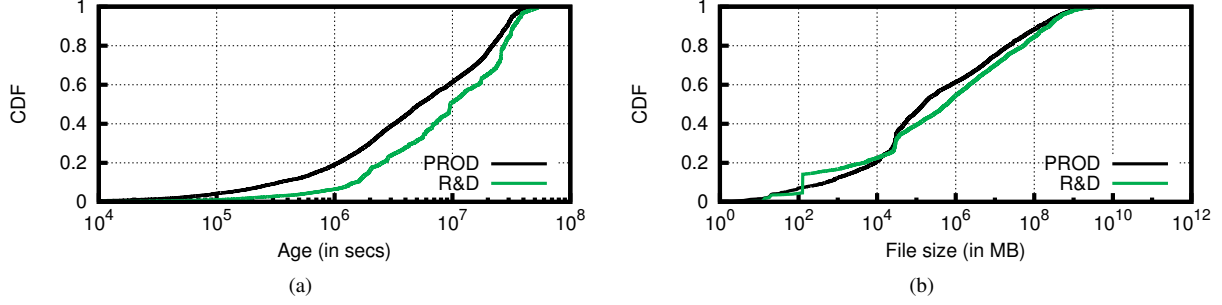


Fig. 11. CDF of the ages and sizes of the files in the namespace snapshot at t_0 . In (b), we binned the file sizes using a 2MB bin; zero-sized files were excluded from the total. At t_0 , the zero-sized files constitute 4% (PROD) and 11% (R&D) of the files in the namespace.

compression, which provides a trade-off between computation and storage/network resources, is a common pattern in Hadoop clusters. Between and 9.27% – 31.65% (PROD – R&D) of the bytes stored are compressed; however, this provides only a lower bound on the percentage of the stored bytes that are compressed because Yahoo! makes heavy use of Hadoop SequenceFiles that may not have an identifiable extension and are—by default in these clusters—compressed.

18: With the exception of compressed files and xml files, no other extension is associated with a significant percentage of the stored files or stored bytes.

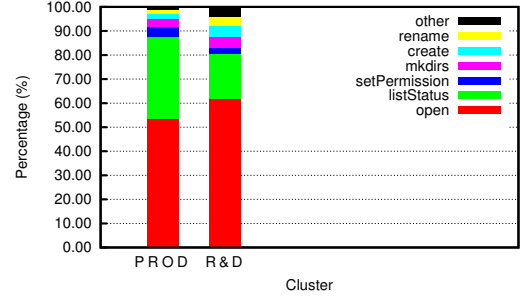


Fig. 12. Percentage of operation types in the 6-month traces.

TABLE IX

STATISTICS OF THE MOST COMMON FILE EXTENSIONS, AS A % OF THE TOTAL NUMBER OF FILES / BYTES IN THE NAMESPACE AT t_0 .

Extension	PROD	R&D
gz	19.00% / 3.80%	11.32% / 8.11%
xml	13.16% / 0.033%	3.28% / 0.003%
bz2	12.18% / 4.29%	18.95% / 15.71%
pig	2.55% / 0.02%	0.24% / 0.007%
dat	1.74% / 0.001%	1.56% / 0.0004%
jar	1.37% / 0.03%	0.24% / 0.007%
proprietary compression	1.28% / 1.42%	2.51% / 7.82%
txt	0.49% / 0.48%	0.18% / 0.017%

G. Percentage of operations

The name node handles the namespace metadata requests, amongst which we have the three operations studied in this paper: create, open and delete. Figure 12 shows the percentage of these and other operations in the 6-month traces. The most common operation is open (55% – 60%), followed by listStatus (ls); together, they account for the vast majority of the operations (80% – 90%). Thus, to be able to satisfy the requests in a timely fashion, the processing of these two types of requests should be handled by the name node in the most efficient manner.

19: The open events account for more than half of the operations issued to the namespace metadata server; open + listStatus, together account for the vast majority of the operations (80% – 90%).

V. RELATED WORK

The workloads of enterprise storage systems [15], web servers [14] and media server clusters [6] have been extensively studied in the past. Big Data clusters have recently

been studied at the job workload level [16, 17], but not at the storage system level. A few recent studies have provided us with some limited insight on the access patterns in MapReduce scenarios [7–9] but these have been limited to those features of interest to the researchers for their specific projects, like block age at time of access [8] and file popularity [7, 9].

Parallel to this work, other researchers did a large scale characterization of MapReduce workloads, including some insights on data access patterns [18]. Their work concentrates on interactive query workloads and did not study the batch type of workload that PROD has. Furthermore, the logs they processed were those of the Hadoop scheduler, and for this reason the authors did not have access to information like age of the files in the system, or when a file is deleted.

Perhaps the work most similar to ours (in approach) is that of Cherkasova and Gupta [6], who characterized enterprise media server workloads. An analysis of the influence of new files and file life span was made, but they did not possess file creation and deletion time stamps, so a file is considered to be “new” the first time it is accessed, and its lifetime “ends” the last time it is accessed. No analysis on the burstiness of requests was made. Their results have been cited in this paper where appropriate, to enable us to contrast MapReduce workloads with a more traditional workload.

Our work complements prior research by providing a better understanding of one type of Big Data workload, filling gaps at the storage level. The workload characterization, key observations and implications to storage system design are important contributions. More studies of Big Data storage workloads and their implications should be encouraged so that storage system designers can validate their designs and

deployed systems can be properly tuned.

VI. DISCUSSION

Existing **file popularity** models and metrics tend to assume (to simplify the model) a somewhat static population. While those models are in practice “wrong” (web sites, words in the English language, etc., appear and disappear in time too), they have proven to be useful when the rate of change of the population is not significant and most population members exist during the time-frame being analyzed.

For the case of the workloads studied in this paper, the analysis presented in § IV-A showed how traditional popularity metrics (e.g., percentage of population that accounts for 90% of the frequency counts—in this case, accesses) can be misleading and make it harder to understand what those numbers imply about the popularity of the population (files). In the analysis presented, the problem arose from the high percentage of short-lived (and thus, infrequently accessed) files. New or adapted models and metrics are needed to be able to better express popularity under these conditions.

The **high rate of change in file population** has some interesting implications in the design of the storage systems: does it make sense to handle the short-lived files in the same way as longer-lived files? Tiered storage systems that combine different types of storage media for different types of files can be tailored to these workloads for improved performance.

While the **burstiness** and autocorrelations in the request arrivals may be a result of typical MapReduce workloads in which multiple tasks are launched within some small time window (all operating on different parts of the same large file or set of related files), a characterization of the autocorrelations is relevant independently of the MapReduce workload that produced them, for the following reasons:

- It allows researchers to reproduce the workload in simulation or real tests without having to use an application workload generator (e.g., Apache GridMix or SWIM [17] for MapReduce). This is useful because current MapReduce workload generators execute MapReduce jobs on a real cluster, which would thus preclude researchers without a large cluster to perform large-scale studies that could otherwise be performed at the simulation level.
- Current MapReduce workload generators (and published models) have overlooked the data access patterns, so their use to evaluate a storage system would be limited.
- Some of the autocorrelations present may also be evident in other Big Data workloads, for example *bag-of-tasks* parallel jobs in High Performance Computing (HPC). If that’s the case, our characterization (and future models that could be proposed) could be useful for designers of storage systems targeted at the HPC community⁷.

VII. CONCLUSIONS AND FUTURE WORK

We presented a study of how two large MapReduce clusters interact with the storage layer. These workloads, from two

large Hadoop (MapReduce) clusters at Yahoo!, have some unique properties that set them apart from previously studied workloads. Their high file churn and skewed access towards young files, among others, should be further studied and modeled to enable designers of next generation file systems to optimize their designs to best meet the requirements of these emerging workloads.

We are working on modeling these workloads and have used our findings to create a workload generator that can mimic the temporal locality access patterns described in this paper [4].

ACKNOWLEDGMENTS

This work was completed during C. Abad’s internship at Yahoo!. R. Campbell and C. Abad are supported in part by AFRL grant FA8750-11-2-0084. Y. Lu is partially supported by NSF grant CNS-1150080.

REFERENCES

- [1] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proc. USENIX OSDI*, 2004, pp. 137–150.
- [2] “Apache Hadoop,” Jun. 2011, <http://hadoop.apache.org>.
- [3] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop Distributed File System,” in *MSST2010*.
- [4] C. Abad, H. Luu, N. Roberts, K. Lee, Y. Lu, and R. Campbell, “Metadata traces and workload models for evaluating Big storage systems,” in *Proc. IEEE UCC*, 2012.
- [5] A. Clauset, C. R. Shalizi, and M. Newman, “Power-law distributions in empirical data,” *SIAM Rev.*, vol. 51, no. 4, Nov. 2009.
- [6] L. Cherkasova and M. Gupta, “Analysis of enterprise media server workloads: Access patterns, locality, content evolution, and rates of change,” *IEEE/ACM Trans. Netw.*, vol. 12, no. 5, 2004.
- [7] G. Ananthanarayanan, S. Agarwal, S. Kandula, A. Greenberg, I. Stoica, D. Harlan, and E. Harris, “Scarlett: Coping with skewed popularity content in MapReduce clusters,” in *Proc. EuroSys*, 2011.
- [8] B. Fan, W. Tantisiroj, L. Xiao, and G. Gibson, “DiskReduce: RAID for data-intensive scalable computing,” in *Proc. PDSW*, 2009, pp. 6–10.
- [9] C. Abad, Y. Lu, and R. Campbell, “DARE: Adaptive data replication for efficient cluster scheduling,” in *Proc. CLUSTER*, 2011.
- [10] W. E. Leland, M. S. Taqqu, W. Willinger, and D. V. Wilson, “On the self-similar nature of Ethernet traffic,” *IEEE/ACM Trans. Netw.*, vol. 2, no. 1, 1994.
- [11] M. E. Crovella and A. Bestavros, “Self-similarity in World Wide Web traffic: Evidence and possible causes,” *IEEE/ACM Trans. on Netw.*, vol. 5, no. 6, 1997.
- [12] K. Park, G. Kim, and M. Crovella, “On the relationship between file sizes, transport protocols, and self-similar network traffic,” in *Proc. ICNP*, 1996.
- [13] G. Casale, E. Z. Zhang, and E. Smirni, “Trace data characterization and fitting for Markov modeling,” *Perform. Eval.*, vol. 67, no. 2, 2010.
- [14] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and Zipf-like distributions: Evidence and implications,” in *Proc. INFOCOM*, 1999.
- [15] Y. Chen, K. Srinivasan, G. Goodson, and R. Katz, “Design implications for enterprise storage systems via multi-dimensional trace analysis,” in *Proc. SOSP*, 2011.
- [16] H. Li and L. Wolters, “Towards a better understanding of workload dynamics on data-intensive clusters and grids,” in *Proc. IPDPS*, 2007.
- [17] Y. Chen, A. Ganapathi, R. Griffith, and R. Katz, “The case for evaluating MapReduce performance using workload suites,” in *Proc. MASCOTS*, 2011.
- [18] Y. Chen, S. Alsbaugh, and R. Katz, “Interactive query processing in Big Data systems: A cross-industry study of MapReduce workloads,” in *Proc. VLDB*, 2012.
- [19] S. Patil, G. Gibson, G. Ganger, J. Lopez, M. Polte, W. Tantisiroj, and L. Xiao, “In search of an API for scalable file systems: Under the table or above it?” in *Proc. USENIX HotCloud*, 2009.

⁷A discussion on whether it is a good idea to have different storage systems for the Internet services community and for the HPC community is out of the scope of this paper. For one particular view on this subject, see [19].